Laboratório de Concorrência · Aula 10

Rômulo · RA 24019338-2 · Turma ESOFTS6C
Gerado em 09/10/2026, 22:13:04 · tempo registrado no cronômetro: 56:16

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação acontece quando uma tarefa depende de outra para continuar. Exemplo: um cozinheiro espera outro terminar de preparar os ingredientes.

Competição acontece quando duas tarefas disputam o mesmo recurso. Exemplo: duas pessoas tentando usar o mesmo caixa eletrônico ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero 2.000.000, pois cada thread incrementa o contador 1.000.000 de vezes. Porém, pode ser menor devido à condição de corrida.

1ª execução (n = 1.000.000)

1.169.274

2ª execução (n = 1.000.000)

1.037.904

3ª execução (n = 1.000.000)

1.923.547

Saída do programa

```
esperado: 2000000
obtido:   1169274
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não. Os valores ficaram abaixo de 2.000.000 e foram diferentes nas três execuções. Isso acontece porque as threads acessam o mesmo contador sem sincronização, causando perda de atualizações.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

As duas threads leem o contador valendo 5. A thread A soma 1 e obtém 6. A thread B também soma 1 e obtém 6. Ambas gravam 6, quando o correto seria 7. Assim, uma atualização é perdida.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Nas duas execuções que fiz, o resultado foi 2.000, o valor correto. Isso é perigoso porque o erro pode não aparecer nos testes com poucas operações, mas surgir em produção com maior concorrência.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis e 4 resultados diferentes: 4, 6, 7 e 8. Apenas uma ordem produz o resultado considerado correto, 8.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA, A lê 3 e calcula 4, mas ainda não grava. B lê 3, multiplica por 2 e grava 6. Depois, A grava seu resultado antigo, 4, sobrescrevendo o valor de B. Assim, a operação de B é perdida.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A versão sem sincronização provavelmente será mais rápida, mas poderá apresentar resultados incorretos. As versões com synchronized e AtomicInteger devem apresentar os valores corretos.

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1898838 em 8 ms
synchronized:      2000000 em 87 ms
AtomicInteger:     2000000 em 39 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger deram o resultado correto, 2.000.000. Elas podem ser mais lentas porque precisam coordenar o acesso das threads ao contador, gerando um custo adicional.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Ele garante a atomicidade de cada saldo individualmente, mas não da transferência completa. Seria necessário utilizar uma transação ou sincronização que protegesse as duas operações juntas.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, porque a thread A segura X e espera Y, enquanto a thread B segura Y e espera X. Isso causa um deadlock.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas

synchronized (X) {
    pausa();
    synchronized (Y) {
        System.out.println("B terminou");
    }
}


A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A bloqueou X e ficou esperando Y. A thread B bloqueou Y e ficou esperando X. Como nenhuma libera o recurso antes de obter o outro, as duas ficam presas.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Porque evita a espera circular entre as threads. Um exemplo seria um sistema bancário que bloqueia duas contas em ordens diferentes durante transferências simultâneas.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

1 trabalhador: aproximadamente 600 ms.

3 trabalhadores: aproximadamente 200 ms.

6 trabalhadores: aproximadamente 100 ms.

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com um trabalhador, as tarefas são executadas uma por vez. Com três ou seis, várias são executadas simultaneamente, reduzindo o tempo total. Não precisa de synchronized porque as tarefas são distribuídas por canais, sem alterar uma variável compartilhada.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!

ch := make(chan int, 1)
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Porque o canal sem buffer precisa de alguém recebendo para concluir o envio, mas não existe outra goroutine recebendo. Com buffer de tamanho 1, a mensagem pode ser armazenada e recebida depois, evitando o bloqueio nesse exemplo.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O compilador deve apresentar um erro porque a thread tenta acessar uma variável local sem transferir sua posse ou utilizar compartilhamento seguro.

Saídas (o erro e o resultado com move)

```
O código original apresenta um erro de compilação relacionado à captura da variável contador
pela closure, como o erro E0373.
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Porque move transfere uma cópia do número para a thread. Ela altera essa cópia, enquanto a variável original de main continua com valor 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o problema pode aparecer durante a execução. Em Rust, o compilador impede determinados acessos inseguros antes de executar. A vantagem é a segurança; o custo é precisar seguir regras mais rigorosas para compartilhar dados.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, C, B e E.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

Porque A e D são executados imediatamente. C é uma microtarefa de Promise, que tem prioridade sobre o callback do setTimeout. Por isso, B aparece depois.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Porque o laço ocupado(300) bloqueou a thread principal por aproximadamente 300 ms. Em um servidor Node.js, cálculos pesados podem atrasar as respostas de outros usuários.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida, permitindo que duas pessoas comprem o mesmo lugar. Para evitar, utilizaria uma transação no banco de dados com bloqueio ou atualização atômica, garantindo que apenas uma compra seja confirmada.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que um programa pode funcionar corretamente várias vezes e ainda apresentar erros dependendo da ordem de execução das threads.